



GRUPO 9 · REQUERIMIENTOS FUNCIONALES

Reportes y Dashboard

Módulo: Panel de control / reportes financieros

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 9
Versión	1.0
Fecha	22 de agosto de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST) y aikuaa-sandbox-web (front)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

Documento derivado del código fuente de aikuaa-sandbox-api y aikuaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre los dos reportes agregados del sandbox sobre el libro de movimientos: el desglose por tipo de movimiento y el resumen general de actividad. Ambos aceptan filtros y devuelven valores calculados por la base de datos, no listados de registros.

Fuera de alcance: Exportación a archivo, gráficos, comparación entre períodos, agregados sobre otras tablas (transferencias, pagos, órdenes) y reportes por moneda: los reportes leen únicamente la tabla de movimientos.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con su propia API key y automatiza escenarios BDD sobre el módulo de su grupo.
Usuario de negocio (fila de usuarios)	Identidad de dominio sobre la que operan los endpoints: es dueño de cuentas, facturas, tarjetas, órdenes, reservas, notificaciones y roles.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo bajo el rol de base de datos qa_api (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz que consume la API a través del proxy /api/proxy/{path}; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v1/reportes/movimientos	RF-G9-01	Desglose por tipo de movimiento, con cantidad y total, filtrable por titular y por rango de fechas.
GET /api/v1/reportes/resumen	RF-G9-02	Resumen general: cantidad, total y fechas del primer y del último movimiento.

2. Precondiciones y acceso

- Toda request a la API debe enviar el header `x-api-key` con una clave existente y con `active = true` en la tabla `public.api_keys`; sin header o con clave inválida/inactiva la respuesta es `401 UNAUTHORIZED` con el mensaje genérico "Invalid or inactive API key".
- Cada API key admite un máximo de **30 requests por minuto** (ventana deslizante). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.
- Los datos de práctica viven en el schema aislado `qa_training` y se cargan con `scripts/seed-data.sql`; la data es determinística, por lo que un escenario puede apoyarse en los registros sembrados.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE`, y las operaciones que "eliminan" hacen baja lógica (`activo = false`).

- En la aplicación web el acceso tiene dos capas: capa 1 la API key (pantalla `/login`, se valida contra `GET /api/v1/roles` antes de guardarse) y capa 2 el usuario de negocio (pantalla `/auth/login`). Con `NEXT_PUBLIC_DEMO_MODE=true` la capa 1 se omite y el proxy inyecta una key demo del servidor.
- Las rutas `/usuarios/*` de la web son la excepción al guard de capa 2: se pueden usar solo con la API key, porque son el punto de partida para obtener un `usuarioId`.
- Los reportes se calculan sobre la tabla `movimientos`, que se carga con los datos sembrados. Ninguna operación de los demás módulos escribe en ella: el volumen de datos es estable entre ejecuciones.
- Las fechas de los filtros se envían como texto en la URL y las interpreta la base de datos; el formato recomendado es `AAAA-MM-DD` o una marca de tiempo ISO 8601.

3. Modelo de datos del módulo

movimientos

Libro de movimientos que atraviesa los distintos dominios del sandbox. Es la única fuente de los dos reportes.

Columna	Tipo / restricción
id	bigserial, clave primaria
usuario_id	bigint, obligatorio, referencia a usuarios(id)
tipo_movimiento	text, uno de transferencia / pago_factura / compra_ecommerce / cargo_tarjeta
monto	numeric(14,2), obligatorio
referencia_id	bigint, opcional y deliberadamente sin clave foránea: según el tipo, apunta a una transferencia, un pago, una orden o una tarjeta
descripcion	text, opcional
created_at	timestampz, por defecto now()

La tabla de movimientos no se alimenta automáticamente: registrar una transferencia, pagar una factura o cerrar una compra **no** agrega un movimiento. Los reportes reflejan únicamente lo sembrado, por lo que un escenario no debe esperar que sus propias operaciones alteren los totales.

4. Requerimientos funcionales

RF-G9-01 — Reporte de movimientos por tipo

GET /api/v1/reportes/movimientos

Devuelve, para cada tipo de movimiento, cuántos movimientos hay y cuál es su importe acumulado, sobre el universo que definen los filtros aplicados.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.
- `desde` (opcional, por query): fecha de inicio del período; incluye los movimientos con fecha igual o posterior.
- `hasta` (opcional, por query): fecha de fin del período; incluye los movimientos con fecha igual o anterior.

Reglas de negocio

- Los tres filtros son opcionales y se combinan con Y lógico; sin ninguno, el reporte abarca todos los movimientos del sandbox.
- El resultado agrupa por `tipo_movimiento` y devuelve una fila por tipo presente en el universo filtrado, ordenada alfabéticamente por tipo.
- Un tipo de movimiento sin registros en el período **no aparece** con cantidad cero: simplemente no figura en el resultado.
- La cantidad se devuelve como número entero y el total como valor numérico exacto (texto con decimales).
- **Las fechas no se validan en la entrada:** viajan como texto y las interpreta la base de datos. Un texto no interpretable produce un error de ejecución.
- Tampoco se valida la coherencia del rango: enviar `desde` posterior a `hasta` es aceptado y devuelve un resultado vacío.
- No hay límite de filas ni paginación, porque la cantidad de tipos posibles es acotada.

Respuesta esperada

- `200 OK` con `data` como arreglo de filas, cada una con `tipo_movimiento`, `cantidad` y `total`.
- Si ningún movimiento cumple los filtros, `data` es un arreglo vacío.

Errores

Código	HTTP	Cuándo ocurre
VALIDATION_ERROR	400	<code>usuarioId</code> presente pero no numérico, cero o negativo.
EXECUTION_ERROR	400	<code>desde</code> o <code>hasta</code> con un texto que la base no puede interpretar como fecha.
UNAUTHORIZED	401	Falta la API key o es inválida.

Fuente: `app/api/v1/reportes/movimientos/route.ts`

RF-G9-02 — Resumen general de movimientos

GET /api/v1/reportes/resumen

Devuelve los indicadores del panel de control: cuántos movimientos hay, cuánto suman, y las fechas del primero y del último.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.

Reglas de negocio

- Con `usuarioId`, los indicadores se calculan sobre los movimientos de ese titular; sin él, sobre todos los movimientos del sandbox.
- Devuelve siempre un único objeto, nunca un arreglo.
- Si no hay movimientos que cumplan el filtro, `cantidad_movimientos` es cero, `total` es cero, y las fechas del primero y del último quedan vacías.
- El total se devuelve como cero (y no vacío) cuando no hay movimientos, para que el panel pueda mostrarlo sin conversiones.
- Este reporte no acepta filtros de fecha: siempre abarca toda la historia del titular.

Respuesta esperada

- `200 OK` con `data` conteniendo `cantidad_movimientos` (entero), `total` (numérico), `primero` (fecha del movimiento más antiguo) y `ultimo` (fecha del más reciente).

Errores

Código	HTTP	Cuándo ocurre
VALIDATION_ERROR	400	<code>usuarioId</code> presente pero no numérico, cero o negativo.
UNAUTHORIZED	401	Falta la API key o es inválida.

Fuente: `app/api/v1/reportes/resumen/route.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo).
- **Coerción de tipos:** los identificadores y montos que llegan por query o por path se convierten de texto a número automáticamente; los campos numéricos de un cuerpo JSON deben enviarse como número, no como texto, salvo que el requerimiento indique lo contrario.
- **Cuerpo JSON malformado:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body".

- **Errores de base de datos:** una violación de restricción (CHECK, UNIQUE, clave foránea) no aborta el servicio: se traduce a `400 EXECUTION_ERROR` con el mensaje que devuelve PostgreSQL.
- **Auditoría:** toda request, exitosa o fallida, deja un registro en `public.sql_audit_log` con la clave usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Idempotencia de las transiciones de estado:** los endpoints que fijan un estado (`bloquear` , `activar` , `confirmar` , `cancelar` , `leer`) escriben el valor destino sin verificar el estado previo, por lo que repetir la llamada devuelve el mismo resultado.

Códigos de error

Código	HTTP	Significado
UNAUTHORIZED	401	Falta el header x-api-key, o la clave no existe o está inactiva.
RATE_LIMITED	429	Se superaron las 30 requests por minuto de esa API key.
VALIDATION_ERROR	400	La entrada no cumple el esquema de la ruta, o el JSON es inválido.
EXECUTION_ERROR	400	La consulta falló en la base de datos (restricción violada, tipo inválido).
NOT_FOUND	404	El recurso indicado no existe, o no está en un estado que admita la operación.
INTERNAL_ERROR	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/{path}` , que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú de navegación muestra únicamente los módulos del grupo del alumno cuando su email figura en el roster del curso; si no figura, muestra los diez módulos. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local.

Pantalla	Qué permite hacer
<code>/reportes</code> — Panel de control	Una sola pantalla con filtros de titular, fecha desde y fecha hasta, la tarjeta de indicadores del resumen y la tabla del desglose por tipo de movimiento. Ejecuta RF-G9-01 y RF-G9-02.

- Los filtros de fecha aplican solo al desglose por tipo: el resumen general no los usa, porque el endpoint no los acepta.
- Con filtros que no arrojan resultados, la pantalla muestra el estado vacío en lugar de una tabla sin filas.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G9-01 — Reporte de movimientos por tipo

- Al pedir el reporte sin filtros, la respuesta es 200 y cada elemento de `data` tiene `tipo_movimiento` , `cantidad` y `total` .

- Los tipos devueltos están ordenados alfabéticamente y no se repiten.
- Al filtrar por un titular, la suma de las cantidades del reporte coincide con la cantidad de movimientos de ese titular.
- Al acotar el período a un rango sin movimientos, la respuesta es 200 con `data` vacío.
- Al enviar `desde` posterior a `hasta`, la respuesta es 200 con `data` vacío, no un error.
- Al enviar `desde=ayer`, la respuesta es 400 `EXECUTION_ERROR`, porque la validación de fechas la hace la base de datos.
- Registrar una transferencia nueva no cambia el resultado del reporte: el libro de movimientos no se alimenta automáticamente.

RF-G9-02 — Resumen general de movimientos

- Al pedir el resumen sin filtro, la respuesta es 200 y `data` es un objeto con las cuatro propiedades del indicador.
- Al pedir el resumen de un titular con movimientos, `data.cantidad_movimientos` coincide con la suma de las cantidades del reporte por tipo del mismo titular.
- Al pedir el resumen de un titular sin movimientos, la respuesta es 200 con `cantidad_movimientos` en cero, `total` en cero y las fechas vacías.
- La fecha de `primero` nunca es posterior a la de `ultimo`.
- Al enviar `usuarioId=0`, la respuesta es 400 `VALIDATION_ERROR`.

8. Anexo normativo (BCP / SEDECO) y brechas

Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Marco del BCP sobre información de operaciones al cliente	El cliente debe poder consultar el detalle y el resumen de sus operaciones por período.	Parcial: existe el desglose y el resumen, pero el resumen no admite acotar el período y el detalle operación por operación no se expone.
Ley 1334/98 de Defensa del Consumidor (SEDECO), derecho a la información	La información debe ser cierta, clara y suficiente para que el consumidor entienda su actividad.	Parcial: los indicadores son claros, pero el libro de movimientos no refleja las operaciones que el propio usuario realiza en el sandbox.
Integridad y trazabilidad de la información contable	Los agregados deben poder reconciliarse contra los registros de origen que los componen.	No implementado: <code>referencia_id</code> no tiene clave foránea y no hay endpoint que permita ir del movimiento a la operación que lo originó.
Ley 6534/2020 de protección de datos personales	Los reportes de un titular solo deben ser accesibles para ese titular.	No implementado: sin filtro, los reportes agregan la actividad de todos los titulares del sandbox.
Presentación de importes en moneda	Los totales deben expresarse indicando la moneda a la que corresponden.	No implementado: <code>movimientos</code> no tiene columna de moneda; los totales suman importes sin distinguirla.

Brechas conocidas del sandbox

- El libro de movimientos no se alimenta con las operaciones del sandbox: transferencias, pagos y órdenes no generan movimientos.
- Los filtros de fecha no se validan en la entrada; un texto inválido llega hasta la base y se reporta como error de ejecución.
- El resumen no acepta rango de fechas, por lo que no puede compararse un período contra otro.
- Un tipo de movimiento sin registros no aparece con cantidad cero, lo que obliga al consumidor del reporte a completar los faltantes.
- No hay columna de moneda: los importes se suman sin distinguirla.
- No hay exportación ni paginación, y `referencia_id` no permite navegar hasta la operación de origen.